



PROJET FPGA : CENTRALE DCC SUR FPGA

Auteurs

AMOURA Zahir
BALDÉ Ibrahima

Encadrant

RHOUNI Amine

29/07/2026

Table des matières

Introduction	2
1 Architecture	2
1.1 Schema Blocs	2
1.2 Table de progression	2
1.3 Div CLK :	3
1.4 DCC_BIT_GENERATEUR :	3
1.4.1 Bit_Timer	3
1.4.2 Machine à États (MEA)	4
1.4.3 Simulation :	5
1.4.4 Validation fonctionnelle et temporelle	5
1.5 Registre à décalage (Decal_registre)	5
1.5.1 Simulation :	6
1.6 TOP	6
1.6.1 Machine à États	6
1.6.2 Description du fonctionnement	7
1.7 simulation finale	8
2 Partie Logicielle	9
2.1 Conception de l'IP	9
2.1.1 Choix de l'Approche 1	10
2.1.2 Définition des registres esclaves	10
2.2 Implémentation de l'application embarquée	11
2.2.1 Fichier dcc_control.c	11
2.2.2 Fichier segment_write.c	11
2.2.3 Fichier test.c	12
2.2.4 Boucle principale et logique de l'IHM	12
2.3 Validation et résultats	12

Introduction

Le protocole DCC (Digital Command Control) est un standard utilisé dans le modélisme ferroviaire pour commander individuellement des locomotives ou des accessoires en modulant la tension d'alimentation de la voie. Les commandes vers les trains sont générées par une Centrale DCC que nous allons implémenter dans le FPGA de la carte Basys, sous la forme d'un système mixte matériel/logiciel. Grâce à une interface utilisateur réalisée à l'aide des boutons de la carte Basys, le FPGA doit générer un signal numérique de commande. Cette commande doit ensuite être amplifiée en courant à l'aide d'une carte Booster, afin d'obtenir un signal suffisamment puissant pour être envoyé sur les rails puis être décodé par les locomotives.

1. Architecture

1.1 Schema Blocs

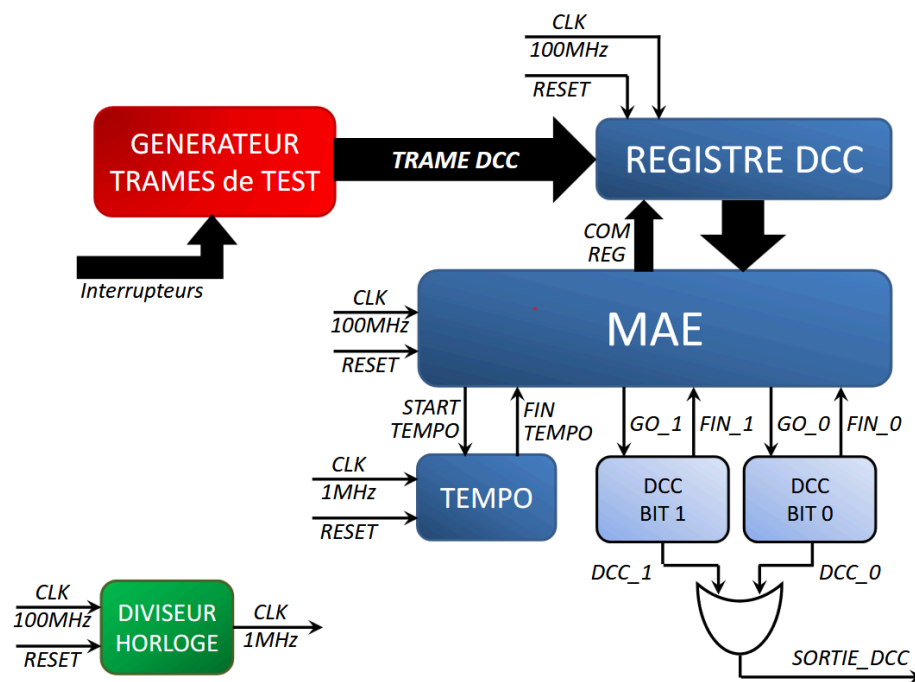


Fig. 1. – Architecture de la Centrale DCC

1.2 Table de progression

Module	Architecture	Test Bench	Validation
Tempo	GIVEN		
Div clock	GIVEN	DONE	DONE
DCC_BIT_GENERATOR	DONE	DONE	DONE
MEA	DONE	DONE	DONE

Registre DC	DONE	DONE	DONE
TOP_DCC	DONE	DONE	DONE

1.3 Div CLK :

fichier source : Diviseur_horloge.vhd

fichier sim : CLK_DIV_SIM.vhd

Pour ce banc de test, la période d'horloge est fixée à $1\mu s$, ce qui correspond à une durée de 1000 ns (pour une fréquence de fonctionnement de 1 MHz).



Fig. 2. – Banc de test

1.4 DCC_BIT_GENERATEUR :

Le module Generateur_Bit_DCC est le bloc d'assemblage (Top-Level) qui interconnecte la machine à états (MEA_generator) et le temporisateur (BIT_TIMER) pour former un générateur complet de bits DCC.

Rôle principal : Peut être configuré pour générer des signaux DCC 1 ou 0, il reçoit l'ordre de générer un bit (Go) et signale quand la transmission est terminée via le signal Fin
Fichier source : Generateur_bit_dcc.vhd

1.4.1 Bit_Timer

Rôle principal : Ce module mesure le temps et génère des temporisations afin de calibrer la durée des bits DCC à transmettre. **Fichier source** : Bit_generateur_timer.vhd

Le module est cadencé par une horloge à **1 MHz**. Le comportement du timer s'adapte dynamiquement selon l'état de l'entrée Bit_Type :

Bit_Type = "1" Le compteur cible un bit DCC à "1". Il incrémente de 0 à 57 (soit une durée de **58 μs**). Une fois cette valeur atteinte, le signal done passe à '1' et le compteur se réinitialise.

Bit_Type = "0" Le compteur cible un bit DCC à "0". La durée requise étant plus longue, il incrémente de 0 à 99 (soit une durée de **100 µs**). À la fin du décompte, done passe à '1' et le compteur se réinitialise.

Signaux de contrôle : L'activation d'un reset global ou d'un signal de remise à zéro synchrone (RAZ = '1') force instantanément le timer à 0 et désactive le signal done.

1.4.2 Machine à États (MEA)

Rôle principal : Cette machine à états gère le séquençage temporel du signal DCC (composé d'une alternance d'un niveau bas et d'un niveau haut). Elle s'appuie sur le signal done du Timer pour piloter ses transitions. **Fichier source :** MEA_Bit_Generateur.vhd

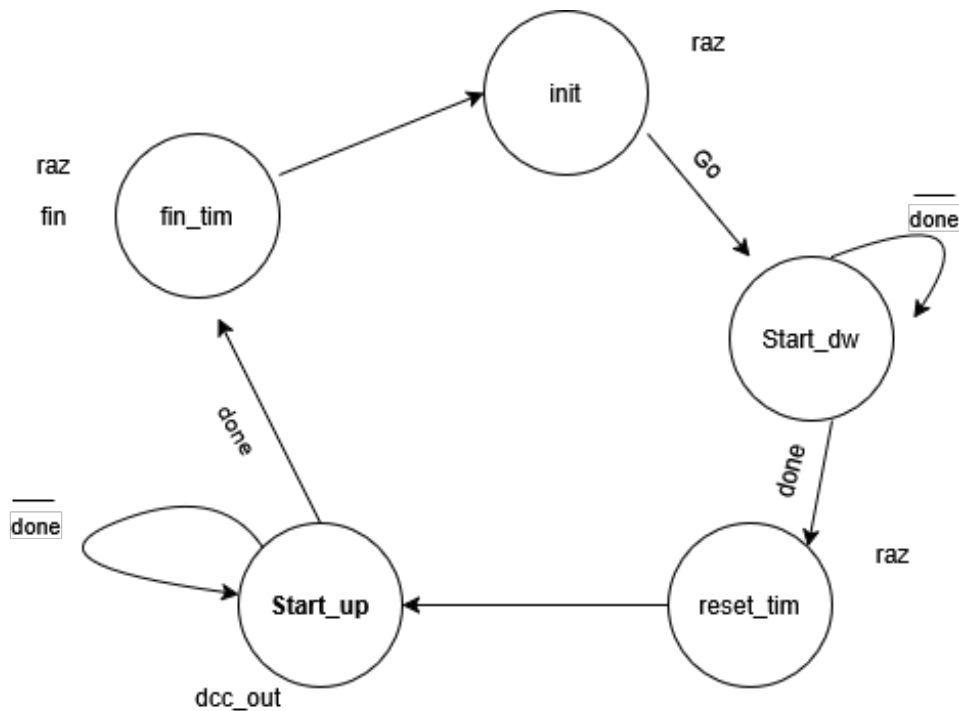


Fig. 3. – MAE

init (État initial / Repos)

- **Actions :** Le timer est maintenu à zéro (raz = '1'), la sortie DCC est désactivée (dcc_out = '0') et le signal de fin est mis à repos (fin = '0').

start_dw (Début de la demi-période basse)

- **Actions :** La machine libère le timer (raz = '0') et force la sortie à l'état bas (dcc_out = '0'). Elle boucle sur cet état tant que le timer est en cours d'opération (done = '0').

reset_tim (Réinitialisation intermédiaire)

- **Transition :** Bascule immédiatement et de façon inconditionnelle vers l'état haut (start_up). applique une impulsion de réinitialisation au timer (raz = '1')

start_up (Début de la demi-période haute)

- **Actions :** Le timer est relancé (raz = '0') et la sortie DCC bascule à l'état haut (dcc_out = '1'). La machine attend la fin de la temporisation (done = '1').

fin_tim (Fin du cycle du bit)

- **Actions :** Émet une impulsion sur le signal de fin (fin = '1')

1.4.3 Simulation :



Fig. 4. – simulation

1.4.4 Validation fonctionnelle et temporelle

L'analyse des chronogrammes de simulation valide la conformité du module `Generateur_Bit_DCC` par rapport aux spécifications temporelles du protocole DCC. On observe deux phases sur la simulation :

- **Génération d'un bit DCC "0" (entre 109 μs et 209 μs) :** La machine à états se trouve initialement dans l'état `start_up`. La durée de cette demi-période haute s'étend précisément de **109,03 μs** à **209,01 μs**, soit une largeur d'impulsion mesurée de **99,98 μs** (ce qui est 0.02 μs pres des 100 μs attendu "0"). À la fin de cette phase, le signal `raz` passe à l'état haut pour réinitialiser le système, et la MEA bascule vers l'état de repos `init`.
- **Génération d'un bit DCC "1" (à partir de 490 μs) :** Suite à une nouvelle impulsion sur le signal `go` alors que l'entrée `bit_type` est positionnée à '1', la machine entame le cycle d'un bit "1". On note la transition séquentielle de l'état `start_dw` vers l'état `start_up` de **557 μs** vers **615,01 μs**, validant ainsi une durée de demi-période de **58 μs**.

Conclusion de la validation : Ces résultats expérimentaux confirment le fonctionnement de la logique du bloc. Le bloc d'assemblage respecte les timings imposés.

1.5 Registre à décalage (Decal_registre)

Rôle principal : Ce module implémente un registre à décalage parallèle-série de **51 bits** (50 downto 0). Son but est de mémoriser une trame DCC complète et de présenter ses bits un à un (successivement) . **Fichier source :** `Decal_registre.vhd`

Le comportement de ce composant est cadencé par l'horloge système `CLK_100mhz` et est gouverné par les signaux de commande suivants :

reset Prioritaire sur toutes les autres signaux. Lorsqu'il est activé à '1', il réinitialise le registre interne (`temp_registre`) à zéro (`others => '0'`).

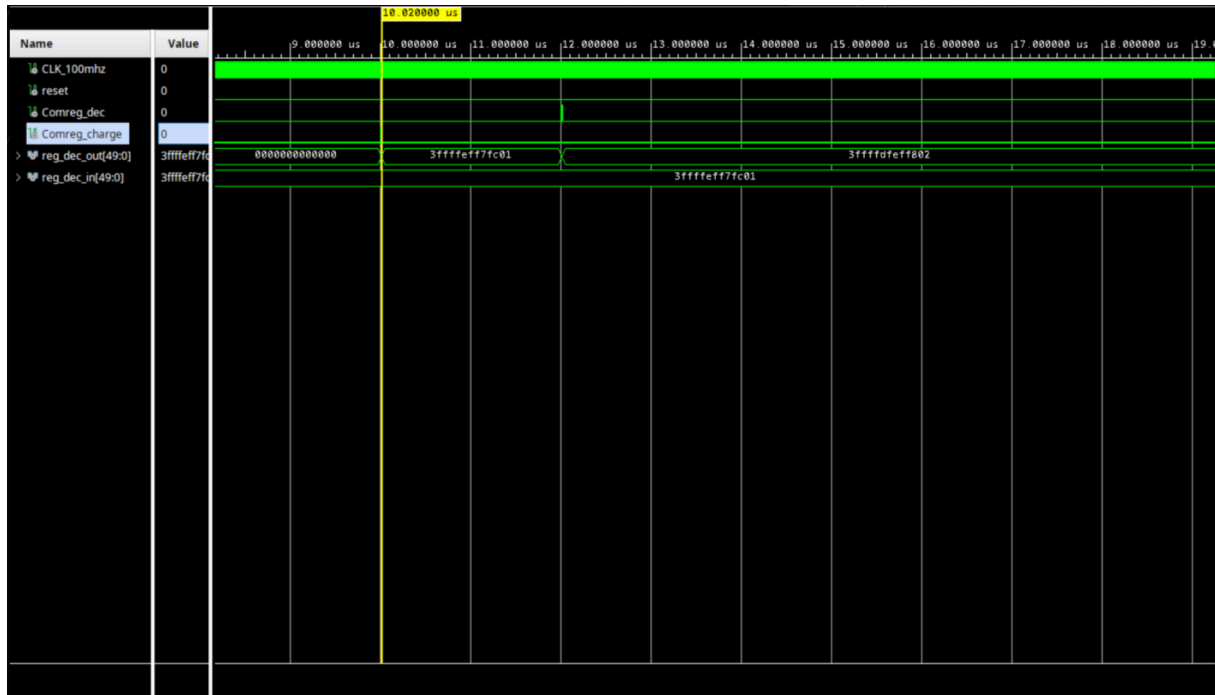
Comreg_charge Commande de chargement parallèle. Sur un front montant d'horloge, si ce signal est actif, la trame complète présente sur l'entrée `reg_dec_in` est mémorisée d'un coup dans le registre.

Comreg_dec Commande de décalage. Sur front montant d'horloge, les bits du registre subissent un décalage d'un cran vers la gauche. Le bit de poids fort (temp_registre(50)) est éjecté, tandis qu'un '0' est injecté du côté du bit de poids faible (temp_registre(0))

```
temp_registre <= temp_registre(49 downto 0) & '0'
```

(Pas de shift sur vhd!)

1.5.1 Simulation :



L'analyse du chronogramme confirme le bon fonctionnement du module Decal_register à travers deux événements clés :

- **Le chargement (à 10 µs) :** la commande s'active, la sortie reg_dec_out prend la valeur de l'entrée reg_dec_in (le signal passe de 0000... à 3ffffeff7fc01). Le chargement parallèle est donc fonctionnel.
- **Le décalage (à 12 µs) :** Une impulsion sur Comreg_dec provoque le décalage des bits vers la gauche. On valide visuellement ce décalage grâce au changement de la valeur hexadécimale, qui passe de ...fc01 à ...f802

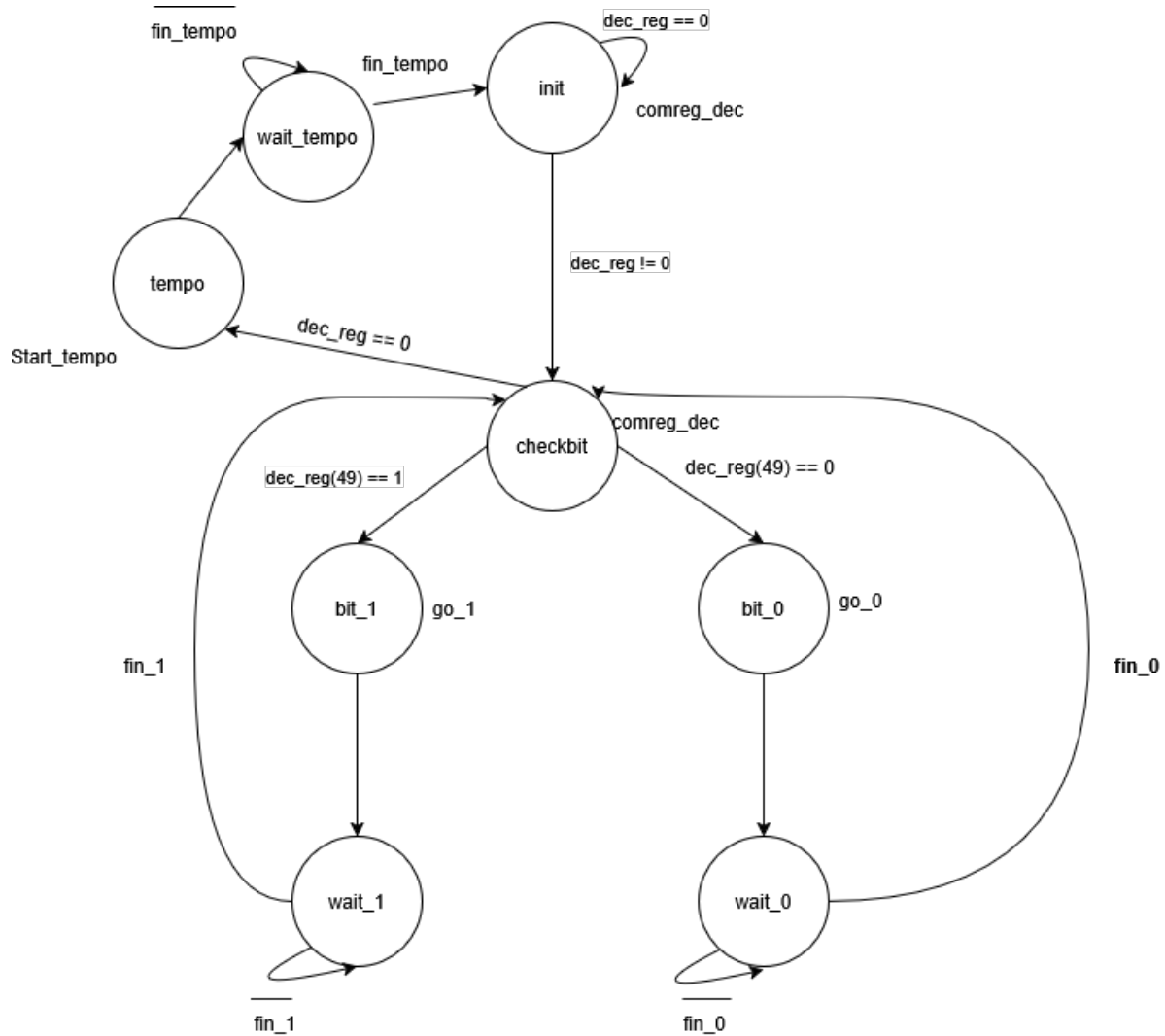
Résultat : Les fonctions de chargement et de décalage sont validées.

1.6 TOP

L'architecture globale intègre tous les blocs fonctionnels au sein d'un module « Top_DCC », piloté par une machine à états..

1.6.1 Machine à États

Rôle principal : Ce module est le cerveau du système (Master FSM). Il lit le contenu du registre de 51 bits, analyse la valeur de chaque bit, et ordonne au générateur DCC d'envoyer soit un bit '1', soit un bit '0' jusqu'à ce que la trame soit entièrement vide. **Fichier source :** MEA.vhd



1.6.2 Description du fonctionnement

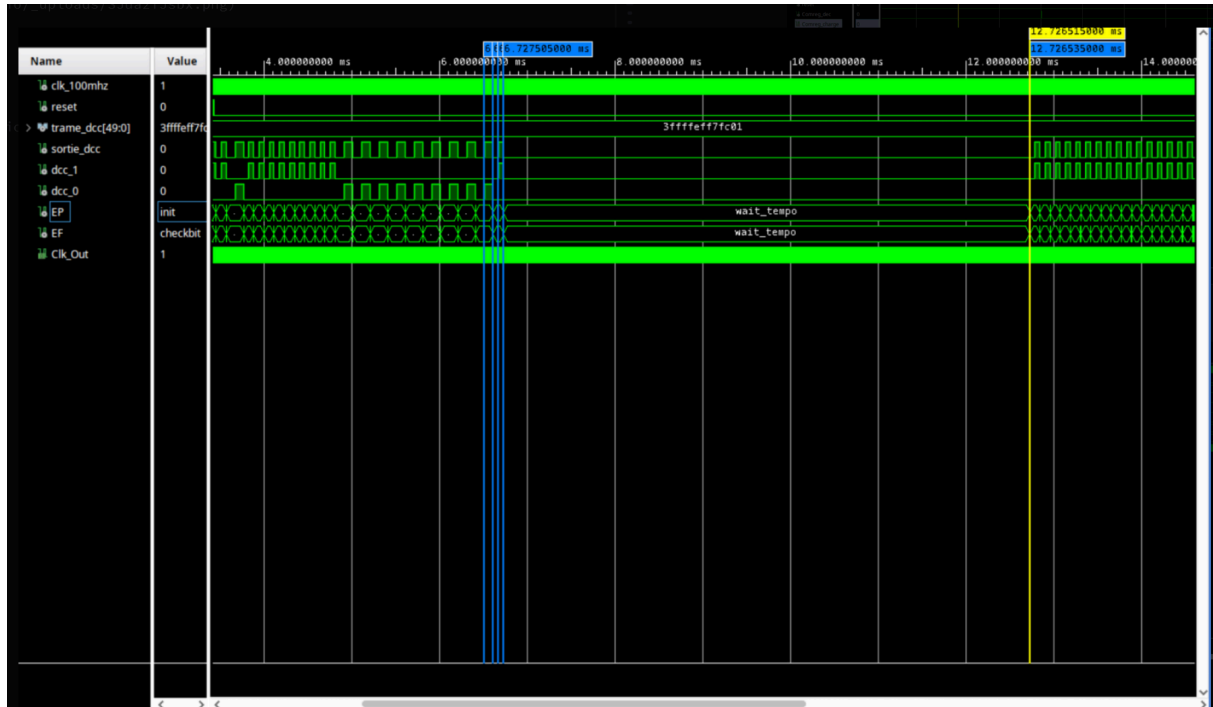
Le comportement de la machine se découpe en 4 grandes étapes logiques :

1. **Initialisation (init) :** La machine active le signal `comreg_charge` $\leq$ '1' pour charger la trame de 51 bits dans le registre, puis bascule vers l'analyse.
2. **Analyse du bit (checkbit) :** La MEA examine le registre. Deux scénarios se présentent :
 - Si le registre est complètement vide (000...000), la trame est finie : elle passe à l'état de temporisation (tempo).
 - Sinon, elle regarde le bit en cours. S'il vaut '1', elle va vers l'état bit1. S'il vaut '0', elle va vers bit0.
 - Ce système fonctionne vu que le dernier bit de la trame est toujours 1.
3. **Génération et Décalage (bit1 / bit0 et wait_1 / wait_0) :**
 - Dès qu'un type de bit est détecté, la machine lance la génération du bit (`go_1` $\leq$ '1' ou `go_0` $\leq$ '1') et un signal au registre pour le décaler (`comreg_dec` $\leq$ '1') pour préparer le bit suivant.
 - Elle attend ensuite dans un état de transition (wait_1 ou wait_0) que le générateur lui renvoie un signal d'acquittement (`fin_1` ou `fin_0`) indiquant que le bit a fini d'être générer.
4. **Temporisation inter-trame (tempo et wait_tempo) :** Une fois les 51 bits envoyés, la machine déclenche une temporisation de 6ms

(`start_tempo` $\leq$ '1') et attend la fin de celle-ci (`fin_tempo` = '1') avant de retourner à l'état init pour charger une éventuelle nouvelle trame.

En résumé : C'est cette machine qui assure l'interface entre les données en mémoire (le registre) et les signaux physiques (les générateurs), bit par bit.

1.7 simulation finale



L'analyse des résultats de simulation confirme le bon fonctionnement de la machine à états globale (MEA) et sa capacité à orchestrer la transmission d'une trame complète.

- **Séquençement des bits :** On valide le comportement temporel fin lors de la génération des bits individuels. Pour un bit '0', l'état start_dw est actif, maintenant dcc_out à la masse. Lors du passage à l'état start_up, la sortie bascule à '1' pendant une durée mesurée de **99,98 µs** (entre **109,03 µs** et **209,01 µs**), ce qui est conforme aux spécifications. Le même comportement est validé pour le bit '1' avec une impulsion conforme de **58 µs**.
- **Gestion inter-trame :** À l'échelle macroscopique, la MEA gère parfaitement la fin de la trame de données frame_dcc. Dès que tous les bits ont été séquentiellement éjectés et transmis (les signaux dcc_1 et dcc_0 repassant au repos), l'état présent EP bascule bien vers l'état wait_tempo. La machine s'y fige pour appliquer la temporisation de sécurité obligatoire avant d'autoriser le chargement de la trame suivante.

Résultat : L'enchaînement des états de contrôle, la détection de fin de trame et le respect des timings physiques du protocole DCC sont entièrement validés.

2. Partie Logicielle

La première partie consistait à écrire la description VHDL, simulé et validé le matériel de la centrale DCC. Dans la partie logicielle, l'objectif est d'intégrer la Centrale DCC au sein d'un système Microblaze.

Dans cette configuration, l'application codée en langage C va servir à gérer l'interface utilisateur. Elle a pour rôle de lire l'état des interrupteurs et des boutons de la carte via des contrôleurs GPIO et de décoder les requêtes de l'utilisateur (vitesse, direction, fonctions secondaires).

Cette section détaillera l'interconnexion globale du système, les choix d'implémentation des registres d'interface de l'IP, ainsi que la logique du code C.

2.1 Conception de l'IP

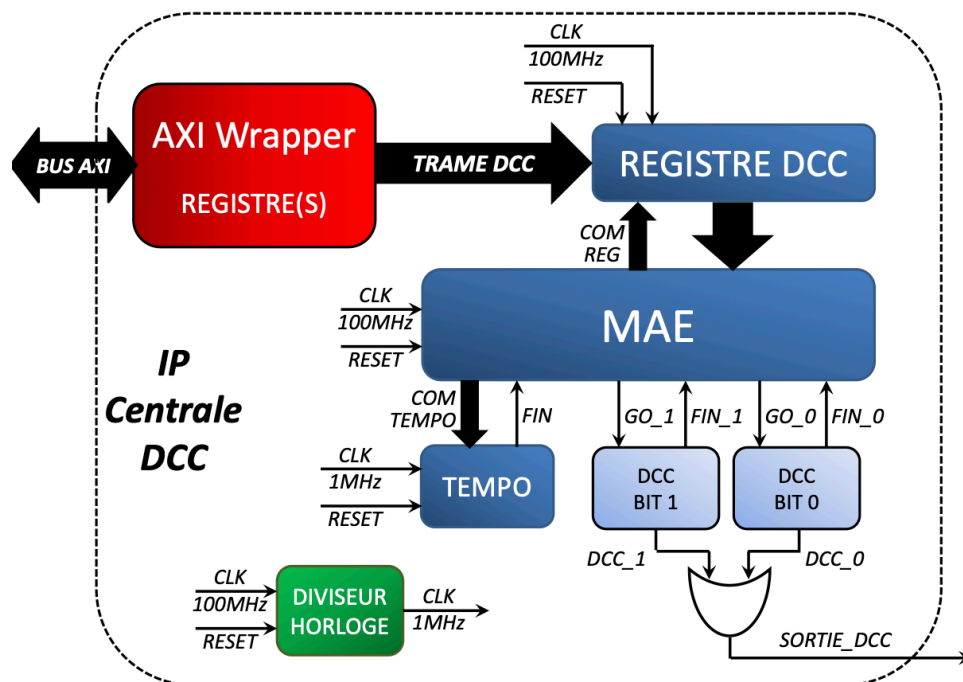


Fig. 5. – IP Centrale DCC

Le seul changement par rapport à l'architecture précédente de la Centrale DCC est que le module de Génération Trames de Test est remplacé par un AXI Wrapper qui permet de rattacher la Centrale DCC au bus AXI du Microblaze.

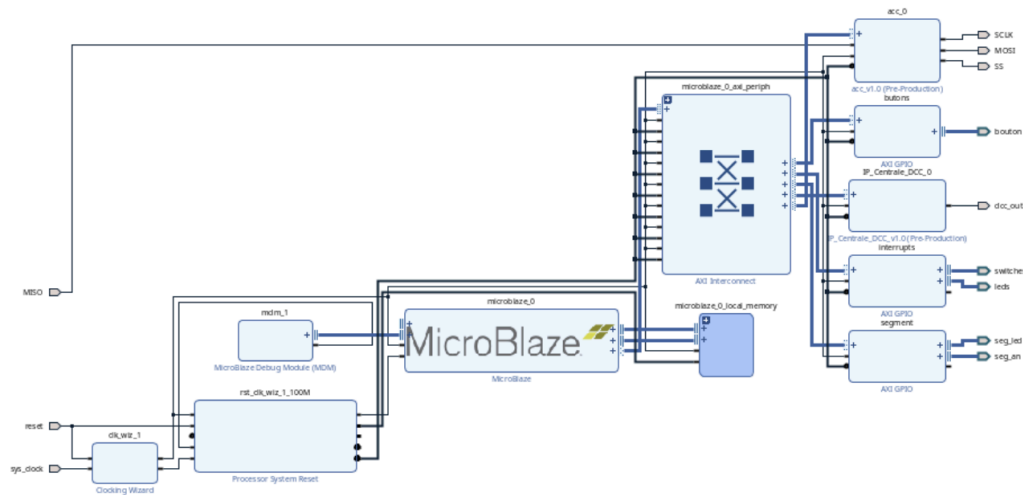


Fig. 6. – Diagramme de blocs

AXI GPIO - Boutons

Le périphérique Boutons est configuré en entrée et permet au MicroBlaze de lire l'état des boutons poussoirs de la carte FPGA.

AXI GPIO - IP_Centrale_DCC_0

L'IP IP_Centrale_DCC_v1.0 est notre périphérique personnalisé qui est chargé de générer les trames DCC destinées au contrôle des locomotives. Elle produit en sortie dcc_out, à partir des commandes transmises par le MicroBlaze via le bus AXI.

AXI GPIO - Interrupts

Ce périphérique est configuré avec deux canaux : un en entrée pour la lecture des interrupteurs et un en sortie pour le contrôle des LEDs de la carte.

AXI GPIO - Segment

Le périphérique Segment pilote l'afficheur 7 segments de la carte via les signaux seg_led et seg_an. L'affichage est géré depuis le logiciel.

2.1.1 Choix de l'Approche 1

Nous avons décidé de préparer la trame DCC complète côté logiciel, avant de la transférer à l'IP. Le MicroBlaze calcule le préambule, les bits de Start/Stop, l'octet de commande et l'octet de contrôle, puis écrit le résultat dans deux registres de 32 bits de l'IP.

Cette approche simplifie l'architecture matérielle de l'IP mais rend le code C plus complexe.

2.1.2 Définition des registres esclaves

La trame DCC fait au maximum 51 bits. Elle est donc répartie sur deux registres 32 bits :

Registre	Offset AXI	Contenu
slv_reg0	0x00	Bits 50 à 19 : préambule (23 bits à 1) + Start bit + adresse (8 bits)
slv_reg1	0x04	Bits 18 à 0 : Start bit + commande (8 bits) + Start bit + contrôle (8 bits) + Stop bit

2.2 Implémentation de l'application embarquée

2.2.1 Fichier dcc_control.c

Dans ce fichier, nous avons implémenté les fonctions de construction et d'envoi des trames DCC. Chaque fonction prend en paramètre l'adresse de base de l'IP, l'adresse de la locomotive et les paramètres de la commande à envoyer.

Commande de vitesse — set_speed_dcc

L'octet de commande est construit selon le format 01DXXXXX défini par le protocole DCC : le bit D fixe le sens de marche et les 5 bits de poids faible encodent la vitesse.

```

1 // Exemple de calcul de la trame de vitesse au bit près en C
2 u8 command = 0x40 | ((forward & 0x01) << 5) | (speed & 0x1F);
3 u8 control = address ^ command; // Calcul du XOR de contrôle
4
5 word0 = (0x7FFFFFF << 9) | (address & 0xFF); // 23 bits de préambule + Start + Adresse
6 word1 = ((u32)command << 10) | ((u32)control << 1) | 0x01; // Commande + Start + Contrôle
   + Stop

```

Commandes de fonctions

Trois fonctions couvrent l'ensemble des groupes de fonctions DCC :

- Function0_4 : gère les fonctions F0 à F4 avec le format 100XXXXX.
- Function5_12 : gère les fonctions F5 à F12 avec le format 101SXXXX.
- Function13_20 : gère les fonctions F13 à F20 sur deux octets .

2.2.2 Fichier segment_write.c

2.2.2.1 Le principe de l'affichage 7 segments

Un afficheur 7 segments est composé de 8 LED (7 segments pour former le chiffre et 1 point décimal). Sur un module à 4 chiffres, relie les segments identiques entre eux (tous les segments "a" sont connectés ensemble, tous les "b", etc.).

Pour déterminer quel chiffre doit s'allumer, on utilise les **anodes** (une broche d'activation par chiffre) :

- Si l'anode d'un chiffre est désactivée, ce chiffre reste éteint, peu importe les signaux envoyés sur les broches des segments.
- **Le principe du multiplexage temporel** : Étant donné que les segments sont partagés, il est impossible d'afficher des chiffres différents simultanément de manière statique. On affiche le premier chiffre, on l'éteint, on active le deuxième, et ainsi de suite.

Le code fourni s'appuie sur la bibliothèque Xilinx GPIO (XGpio). Il est structuré autour de deux canaux : le **Canal 1** contrôle l'état des segments (les LED à allumer) et le **Canal 2** gère la sélection des anodes (le chiffre à activer). Le système utilise une logique **Active-Low** (un état bas 0 allume, un état haut 1 éteint).

- **write_digit(segment, number)** : Traduit un chiffre unique (de 0 à 9) en son équivalent binaire (code hexadécimal) pour configurer l'allumage des LED des segments physiques.
- **select_digit(segment, digit_index)** : Active l'anode du chiffre choisi (de 0 à 3) en la mettant à la masse (logique inversée) et désactive toutes les autres pour cibler l'affichage.
- **write_number(segment, total_number, del)** : Décompose un nombre entier (jusqu'à 9999) en ses quatre chiffres individuels et gère la boucle de multiplexage temporel avec une temporisation pour afficher le nombre complet.

- **write_letter_num(segment, letter_idx, number, delay_timer)** : Permet un affichage mixte en appliquant un motif de lettre spécifique sur le chiffre le plus à gauche et un nombre à trois chiffres sur le reste de l'afficheur.

2.2.3 Fichier test.c

Dans le fichier test.c on retrouve trois structures de données :

- SpeedSelect : l'adresse, la vitesse et le sens de marche d'un train.
- FunctionSelect : l'adresse, le type de groupe de fonctions , le masque de fonction et le groupe
- Buton : gère l'état d'un bouton avec un flag d'événement, un identifiant de bit GPIO et un état d'appui courant.

is_pressed permet la détection des appuis .

2.2.4 Boucle principale et logique de l'IHM

La boucle principale lit en permanence l'état des interrupteurs et des boutons, et implémente une machine à états implicite basée sur les flags des boutons :

Bouton	Mode activé	Validation
Gauche	Saisie de l'adresse de la locomotive	Centre
Haut	Saisie de la vitesse et du sens	Centre
Bas	Sélection d'une fonction F0-F20	Centre
Droite	Envoi immédiat de la fonction sélectionnée	—

Quand on veut sélectionner une fonction, les boutons Haut et Droite permettent d'incrémenter ou de décrémenter le numéro de fonction affiché. En appuyant sur le bouton Centre, le code détermine le groupe de fonctions et construit le masque binaire correspondant.

2.3 Validation et résultats

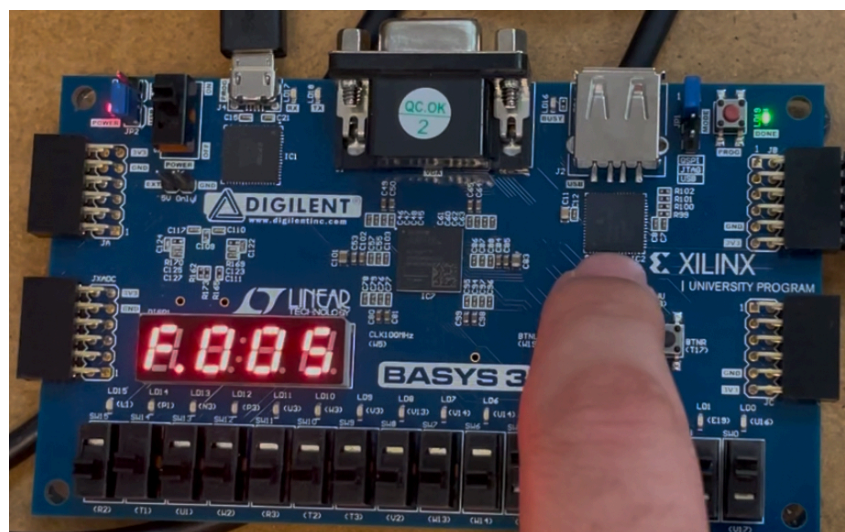


Fig. 7. – Sélection de la fonction numéro 5

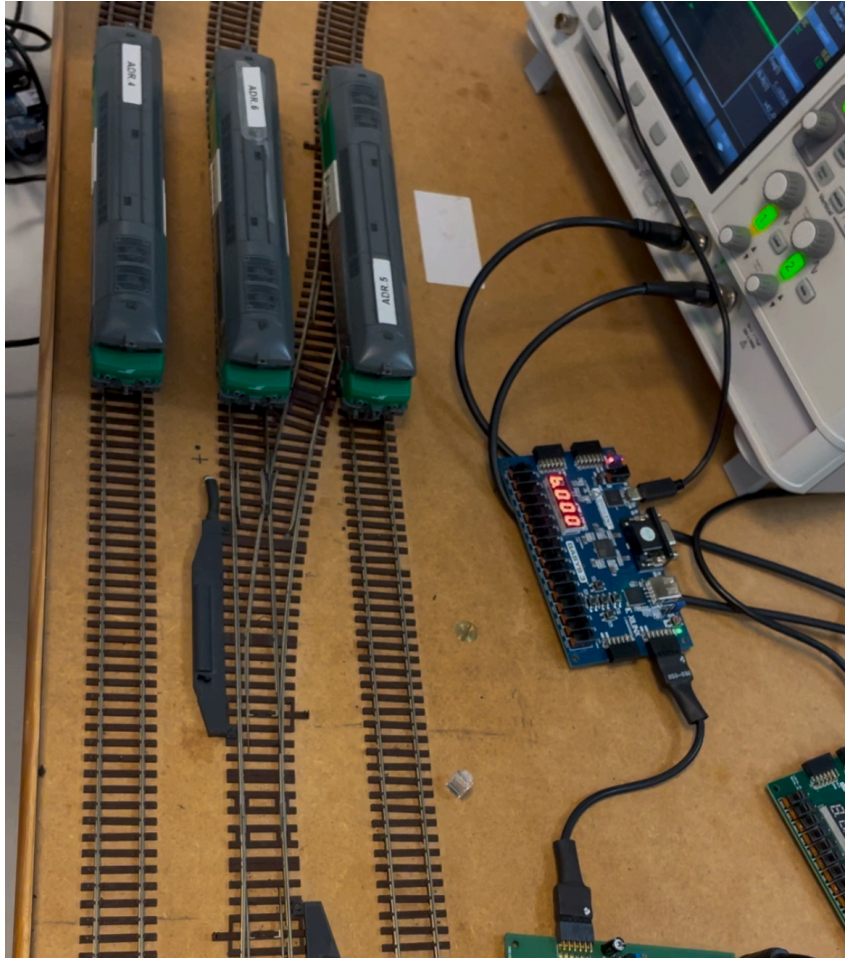


Fig. 8. – Validation avec 3 locomotives sur la maquette

Nous avons testé notre système avec plusieurs locomotives simultanément sur la maquette. Chaque locomotive répond correctement aux commandes de vitesse et de fonctions qui lui sont adressées, confirmant le bon fonctionnement du ciblage par adresse DCC.